Algoritmer Formelsamling

1 Notation

$O(f(n))$: Upper bound - Rate of growth for worst-case input of size n
$\Omega(f(n))$: Lower bound - Rate of growth for best-case input of size n
$\Theta(f(n))$: Tight bound - When $O(f(n)) = \Omega(f(n))$
$o(g(n))$: Strictly upper bound - $f(n)$ grows slower than $g(n)$
$\omega(g(n))$: Strictly lower bound - $f(n)$ grows faster than $g(n)$

When finding complexity, only the fastest growing term is kept, and constants are dropped. Only Θ needs be tight, the others just have to be upper/lower bounds, even if they are far from tight.

1.1 – Hierarchy of growth rates

$$1 < \lg\lg n < \lg n < \lg^k n < n^{k\in]0,1[} < n < n\lg n < n^2 < n^3 < 2^n < n! < n^n$$

Note: $\sqrt{n} = n^{1/2}$

1.2 – Ackermann functions

$A_m(n)$: The Ackermann function is practically infinite for $n > 4$.
$\alpha(n)$: The Inverse Ackermann function is ≤ 4 for all practical values n.

Use

Example of the Ackermann function:
Given that $A_2(1) = 7$:

$$A_3(1) = A_2(A_2(1)) = A_2(7) = 2^{7+1} - 1 = 2047$$

This means that:

$$A_4(1) = A_3(A_3(1)) = A_3(2047) = A_2(A_2(...A_2(1)...)) \text{ [2047 times]} \approx \infty$$

2 Search

Unsorted List: Linear Search[1] Sorted List: Binary Search

2.1 – Linear search

Go through each item from start to end.

Time: $O(n)$ $\Omega(1)$

2.2 – Binary search

Only works on sorted lists.
Start at the middle of the list, and check if it is: the target, lower or higher.

- Target: Return the index of the target.
- Lower: discard upper half and repeat the process on the lower half.
- Higher: discard lower half and repeat the process on the upper half.

Time: $O(\lg n)$ $\Omega(1)$

```
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = (mid + 1)
        else:
            right = (mid - 1)
    return -1
```

3 Sorting

3.1 – Merge Sort

Works by splitting the list into halves recursively until size 1. Then it merges the halves back together in the correct order.

Time: $\Theta(n \lg n)$

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])
    return merge(left_half, right_half)
```

[1] Faster than sorting the list and then doing a binary search if you only need to search once. Otherwise it is best to sort the list and then do a binary search.

4 Stacks

FIFO (First In First Out) data structure.

1. **push(x)**: Add element x to the top of the stack.
2. **pop()**: Remove and return the top element of the stack.
3. **isEmpty()**: Return true if the stack is empty, false otherwise.

4.1 – Implementations

Array-based implementation

Make an array of size `N` and a variable `t` to keep track of the top index.

push(x): If `t < N-1`, set `t++` and `arr[t] = x`.
pop(): If `t != -1`, return `arr[t]` and `t--`.
isEmpty(): Return true if `t < 0`, false otherwise.

Time: $\Theta(1)$ **Space:** $\Theta(N)$

Linked list-based implementation

Dynamic data structure. More efficient than array-based implementation.

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None
class Stack:
    def __init__(self):
        self.top = None
    def push(self, x):
        new_node = Node(x)
        new_node.next = self.top
        self.top = new_node
    def pop(self):
        if self.top is None:
            return None
        value = self.top.value
        self.top = self.top.next
        return value
    def isEmpty(self):
        return self.top is None
```

Time: $\Theta(1)$ **Space:** $\Theta(n)$

Dynamic array-based implementation

Make array `A` size `N`, array `B` size `2N` and variable `t` for top index. *Push* adds to `A` and copies 2 from `A` to `B`. When `A` is full, set `A = B` and make a new array `B` of size `4N`. *Pop* removes from `A` and removes the same index from `B`. When `A` $<$ `N/4`, set `B = A` and make a new array `A` of size `N/2`.

Time: $\Theta(1)$ **Space:** $\Theta(n)$

5 Queues

LIFO (Last In First Out) data structure.

1. **enqueue(x)**: Add element x to the end of the queue.
2. **dequeue()**: Remove and return the front element of the queue.
3. **isEmpty()**: Return true if the queue is empty, false otherwise.

5.1 – Implementations

Array-based implementation

Make an array of size `N` and two variables `f` and `b` to keep track of the front and back indices. The queue snakes around the array using modulo.

enqueue(x): if `(b+1) % N != f` set `arr[b] = x` and `b = (b+1) % N`.

dequeue(): if `f != b`, return `arr[f]`, set `f = (f+1) % N`.

isEmpty(): Return true if `f == b`, false otherwise.

Time: $\Theta(1)$ **Space:** $\Theta(N)$

Linked list-based implementation

Dynamic data structure. More efficient than array-based implementation.

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None
class Queue:
    def __init__(self):
        self.front = None
        self.back = None
    def enqueue(self, x):
        new_node = Node(x)
        if self.back is None:
            self.front = self.back = new_node
        else:
            self.back.next = new_node
            self.back = new_node
    def dequeue(self):
        if self.front is None:
            return None
        value = self.front.value
        self.front = self.front.next
        if self.front is None:
            self.back = None
        return value
    def isEmpty(self):
        return self.front is None
```

Time: $\Theta(1)$ **Space:** $\Theta(n)$

Dynamic array-based implementation

Can be done with multiple arrays like with stacks with same complexity.

6 Graphs

Graphs have `n` vertices V and `m` edges E.

1. **Adjacent(u,v)**: Return wheter there is an edge from vertices `u` to `v`.
2. **Neighbors(u)**: Return list of all vertices `v` with edges from `u`.
3. **Insert(v,u)**: Insert an edge from vertex `v` to vertex `u`.

6.1 – Representations

Adjacency matrix

`n` $\times$ `n` boolean matrix where `mat[u][v]` is true if there is an edge from vertex `u` to vertex `v`. Efficient for dense graphs.

Time: $\Theta(1, n, 1)$ **Space:** $\Theta(n^2)$

Adjacency list

Array of size `n` containing linked lists of neighbors.
Efficient for sparse graphs.[2]

Time: $\Theta(\deg(v))$ **Space:** $\Theta(n + m)$

6.2 – Search

Depth-first search (DFS)

1. Pick a node.
2. Add all unseen neighbors to stack.
3. Pop from stack and repeat until stack is empty.

Nodes can be unmarked, seen, or visited. Unmarked nodes are not in stack, seen nodes are in stack, and visited nodes have been popped from stack.

```python
def dfs(graph, start): # Adjacency list and starting vertex.
    visited = set()
    stack = [start]

    while stack:
        vertex = stack.pop()
        if vertex not in visited: # Since we uncritically add neighbors to the stack.
            visited.add(vertex)
            stack.extend(graph[vertex] - visited) # Add all unvisited neighbors to stack.

    return visited
```

Time: $\Theta(n + m)$ **Space:** $\Theta(n)$

[2] Almost all real-world graphs are sparse

Breadth-first search (BFS)

Like DFS but uses a queue instead of a stack. Finds the shortest path in an unweighted graph.

```python
def bfs(graph, start):
    visited = set()
    queue = [start]

    while queue:
        vertex = queue.pop(0)
        if vertex not in visited:
            visited.add(vertex)
            queue.extend(graph[vertex] - visited) # Add unvisited neighbors to the queue.

    return visited
```

Time: $\Theta(n + m)$ **Space:** $\Theta(n)$

6.3 – Applications

Assuming we use adjacency lists.

Count connected components

Run DFS or BFS on any unvisited vertex and count how many times you have to run it to visit all nodes.

Time: $\Theta(n + m)$

Shortest path

In an unweighted graph, a BFS tree finds a shortest path[3] from the starting vertex to all other vertices.

Time: $\Theta(n + m)$

Distance

By giving each vertex a distance field, we can find the distance from the starting vertex to all other vertices.

[3]There can be multiple shortest paths, BFS will find one of them

```python
def bfs_with_distance(graph, start):
    visited = set()
    queue = [start]
    start.dist = 0 # Set distance of starting vertex to 0.

    while queue:
        vertex = queue.pop(0)
        if vertex not in visited:
            visited.add(vertex)
            for neighbor in graph[vertex]:
                if neighbor not in visited:
                    queue.append(neighbor)
                    neigbor.dist = vertex.dist + 1
```

Time: $\Theta(n+m)$

Bipartiteness

A graph is bipartite if it can be colored with two colors such that no adjacent vertices are the same color. $\implies$ the graph has no even-length cycles.

```python
def is_bipartite(graph):
    bfs_with_distance(graph, start)
    # Run BFS to find distance of all vertices from starting vertex.
    for vertex in graph:
        for neighbor in graph[vertex]:
            if vertex.dist % 2 == neighbor.dist % 2:
                # If two adjacent vertices are both odd/even distance from the starting
                ↪ vertex, then they are the same color and the graph is not bipartite.
                return False
```

Time: $\Theta(n+m)$

7 Directed Graphs

7.1 – Representations

Like with undirected graphs in 6.1, we can use matrices and linked lists.

7.2 – Topological sort

A topological sort is a sort such that all vertices only point to vertices that come after them in the sort.

To execute a topological sort:

1. Find vertex with in-degree 0. Remove it from the graph and add it to the sort.
2. Repeat until all vertices are removed.

Implementations

1. Reverse Graph Take the adjacency list and construct a reverse graph. Then go through the array to find vertices with in-degree 0 and add them to the sort. Remove them from the graph and repeat until all vertices are removed.

Time: $\Theta(n^2)$

2. In-degree array Maintain the in-degree of each vertex in an array. Initially, add all vertices with in-degree 0 to a queue. Then repeatedly pop from the queue, add the vertex to the sort, and decrease the in-degree of its neighbors. If any neighbor's in-degree becomes 0, add it to the queue.

Time: $\Theta(n+m)$

DAGs

A topological sort is possible $\iff$ the graph is a DAG.[4]

7.3 – Automata

Automata are directed graphs where edges have values, and a path from a starting vertex to an ending vertex represents a string of values.
$a, b, c, \cdots$ are values, and ϵ means no value.
Regular expressions can represent all possible paths on the automata.
Components of regular expressions:

$a \cdot b$: a followed by b.

$a|b$: a or b.

a^* : a repeated 0 or more times.

a^+ : a repeated 1 or more times.

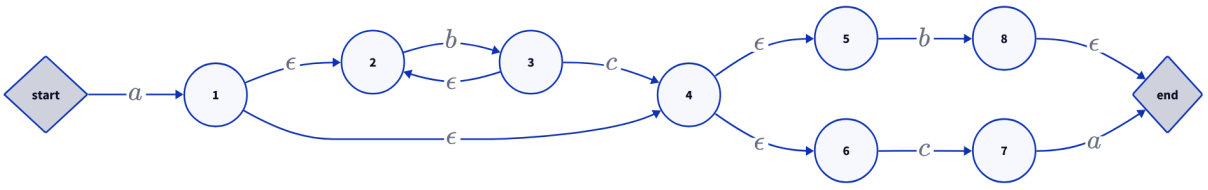

Figure 1: Automata with the regular expression $a \cdot ((b^* \cdot c)|\epsilon) \cdot (b|(c \cdot a))$

7.4 – Strongly connected components

A strongly connected component is a maximal subgraph where there is a path from any vertex to any other vertex.

[4] directed acyclic graph

8 Heaps and Priority Queues

Priority queues support the operations:

1. **Max()**: Return elment with largest key.
2. **ExtractMax()**: Remove and return element with largest key.
3. **IncreaseKey(x, k)**: Increase the key of element x to k.
4. **Insert(x)**: Insert element x into the priority queue.

8.1 – Heaps

Heaps are *almost complete* binary trees[5] where each node has a key that is greater than or equal to the keys of its children.

Operations

BubbleUp(x): If the key of x is greater than the key of its parent, swap x with its parent and repeat until the heap property is restored.
BubbleDown(x): If the key of x is less than the key of one of its children, swap x with the child with the largest key and repeat until the heap property is restored.

Time: $O(\log n) \quad \Omega(1)$

Representations

Linked list: Each node has pointers to its parent and children. Allows arbitrary size.
Array: The root is at index 1, the left child of i is $2i$, the right child $2i+1$, and the parent is $\lfloor i/2 \rfloor$. More space efficient than linked list.

8.2 – Building heaps

To turn an array into a heap we use *Bottom-up construction*. For the first $[1, \lfloor n/2 \rfloor]$ elements, we call **BubbleDown(i)**, starting from $\lfloor n/2 \rfloor$ and going down to 1.

Time: $O(n) \quad \Omega(1)$

8.3 – Using heaps for priority queues

Below is baed on heaps as arrays. Logic is similar for linked lists.

[5] All nodes have two children except for the last level, which is filled from left to right

```
def max(heap):
    return heap[1]

def extract_max(heap):
    max_element = heap[1]
    heap[1] = heap[length] # Move last element to root.
    length -= 1 # Remove last element.
    bubble_down(heap, 1) # Restore heap property.
    return max_element

def increase_key(heap, i, k):
    heap[i] = k
    bubble_up(heap, i) # Restore heap property.

def insert(heap, k):
    length += 1
    heap[length] = k
    bubble_up(heap, length) # Restore heap property.
```

8.4 – HeapSort

To sort an array, build a heap and repeatedly run `extractMax()`.
This has the advantage of being an in-place sorting algorithm.

Time: $\Theta(n \lg n)$

9 Union Find

Union find is a data structure that keeps track of disjoint subsets. It supports:

1. `Init(n)`: creates `n` disjoint subsets.
2. `Union(a,b)`: merges the subsets containing `a` and `b`.
3. `Find(a)`: returns the representative of the subset containing `a`.

Representatives are one value in the subset that identifies it. In tree representations, the representative is the root of the tree.

9.1 – Enhancements

Union by rank makes smaller trees point to large trees.
Path compression makes all nodes on a path point directly to the root after a find operation.
Both keep the trees flat, therefore faster.

```
def init(n):
    return [Node(i) for i in range(n)] # Create n nodes, each in their own subset.

def find(x):
    if x.parent != x:
        x.parent = find(x.parent) # Path compression
    return x.parent

def union(a, b):
    a_root = find(a)
    b_root = find(b)
    if a_root == b_root:
        return
    # Weighted union by rank
    elif a_root.rank < b_root.rank:
        a_root.parent = b_root
    elif a_root.rank > b_root.rank:
        b_root.parent = a_root
    else:
        b_root.parent = a_root
        a_root.rank += 1
```

For any sequence of `m` union and find operations **Time:** $O(m \cdot \alpha(n)) \quad \Omega(m)$

10 Minimum Spanning Trees

Given a weighted undirected graph, a MST is a tree connecting all vertices with the least total weight.
To computer a MST, we can use adjacency matrices, adjacency lists, or edge lists.

10.1 – Prim's algorithm

Given an adjacency list or matrix and starting vertex, at each step add the edge with the least weight that connects a vertex in the tree to a vertex outside the tree. Repeat until all vertices are in the tree.
To implement, maintain a priority queue of vertices outside the tree, where the priority is the weight of the least weight edge connecting the vertex to a vertex in the tree. At each step, pop the vertex with the least priority and add it to the tree. Then update the priorities of its neighbors.

10.2 – Kruskal's algorithm

Given an edge list, sort the edges by weight. Then go through the edges in order and add them to the tree if they connect two vertices that are not already connected. Repeat until all vertices are in the tree.
To implement, maintain a union-find data structure to keep track of which vertices are in the same tree. Each time an edge is added, union the two vertices.
Given a graph with `n` vertices and `m` edges, both are

Time: $\Theta(m \lg n)$

11 Shortest Paths

All subpaths within a shortest path are also shortest paths.

Notice: MSTs *don't* always contain a shortest path between vertices!

11.1 – Relaxing

Relaxing an edge (u, v) means updating the distance estimate of v:

$$\text{est}(v) := \min(\text{est}(v), \text{est}(u) + \text{wgt}(u, v))$$

11.2 – Dijkstra

Works only when all edge weights are non-negative.

1. Set distance estimate of all vertices to ∞, except source which is 0.
2. While there are unvisited vertices:
   (a) Pick unvisited vertex with smallest distance estimate, call it u.
   (b) Mark u as visited.
   (c) Relax all edges outgoing from u.

Dijkstra's algorithm uses a priority queue for unvisited vertices.

Time: $\Theta((|V| + |E|) \lg |V|)$

11.3 – Bellman-Ford

Like Dijkstra's, slower but also works with negative edge weights.
Works by relaxing every edge $|V| - 1$ times. Can be made slightly faster by only relaxing edges that were relaxed in the previous iteration.

Time: $\Theta(|V| \cdot |E|)$

11.4 – DAG

For a DAG, shortest paths are found by processing in topological order.

1. Set distance estimate of all vertices to ∞, except source which is 0.
2. Add the next vertex in topological order to visited.
3. Relax all edges outgoing from the vertex.

Assuming we first have to topologically sort the graph **Time:** $\Theta(|V| + |E|)$

11.5 – Loops

If a graph has a negative weight cycle, there is no shortest path between vertices connected to the cycle.

12 Search Trees

Search trees need to maintain a set that supports:

1. `Search(k)`, return `x` if `x.key = k`, else `NIL`
2. `Insert(x)`, add `x` to the set
3. `Delete(x)`, remove `x` from the set

12.1 – Binary Search Trees

BST's are binary trees where all keys are stored in *symmetric order*:

- For any node `x`, all keys in the left subtree are less than `x.key`
- For any node `x`, all keys in the right subtree are greater than `x.key`

BST traversal by hand

Starting at the root, go countrclockwise around the tree, sticking close to the tree.

- Pre-order: Add to list when on the left of the node
- In-order: Add to list when under the node
- Post-order: Add to list when on the right of the node

They support all the operations in **Time:** $O(\text{height}) \quad \Omega(1)$

Height is on average $\Theta(\lg n)$, but worst case $O(n)$.

12.2 – 2-3 Trees

2-3 trees always have height $\Theta(\lg n)$. Nodes are 2- or 3-nodes.
All leaves are at the same depth, so the tree is perfectly balanced.
2-node: 1 key and 2 children. *3-node*: 2 keys and 3 children.

Search, Insert, Delete have **Time:** $\Theta(\lg n)$

Inserting

To insert, search for the key until you reach a leaf.

- Node is 2-node $\Rightarrow$ add key and turn into 3-node
- Node is 3-node $\Rightarrow$ add key and turn into 4-node, split into two 2-nodes and insert middle key to parent. (bubble up)
- Node is 3-node and root $\Rightarrow$ add key and turn into 4-node, split into two 2-nodes and make middle key the new root.

Deleting

TBD

13 Segment Trees

Segment trees require the number of elements to be a power of 2. If not, pad with ∞.

Notice: Unless otherwise specified, segment trees are minimum trees.

13.1 – Segment Tree as Heap Array

If the number of elements is static, an array is efficient.
Node j has children $2j$ (left) and $2j + 1$ (right) and parent $\lfloor j/2 \rfloor$.

Building a Segment Tree as a Heap Array

1. Pad input until length is a power of 2.
2. Make new array of length $2n$. (note the first element is not used)
3. Make all indexes i be $n + i$ in the new array.
4. For all other indexes i, `A[i] = min(A[2i], A[2i+1])`.

Operations on Segment Trees as Heap Arrays

To increase element `i`, increase `A[n+i]` and update all ancestors.

Binary Search Tree

```python
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.parent = None

def search(root, k):
    if root is None or root.key == k:
        return root
    elif k < root.key:
        return search(root.left, k)
    else:
        return search(root.right, k)

def insert(root, x):
    y = None
    current = root
    while current is not None:
        y = current
        if x.key < current.key:
            current = current.left
        else:
            current = current.right
    x.parent = y
    if y is None:
        root = x  # Tree was empty
    elif x.key < y.key:
        y.left = x
    else:
        y.right = x

def delete(x):
    if x.left is None:
        transplant(x, x.right)
    elif x.right is None:
        transplant(x, x.left)
    else:
        y = minimum(x.right)
        if y.parent != x:
            transplant(y, y.right)
            y.right = x.right
            y.right.parent = y
        transplant(x, y)
        y.left = x.left
        y.left.parent = y

def transplant(u, v):
    if u.parent is None:
        root = v
    elif u == u.parent.left:
        u.parent.left = v
    else:
        u.parent.right = v
    if v is not None:
        v.parent = u.parent

def minimum(node):
    while node.left is not None:
        node = node.left
    return node
```